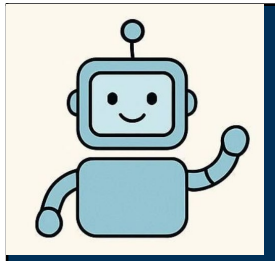
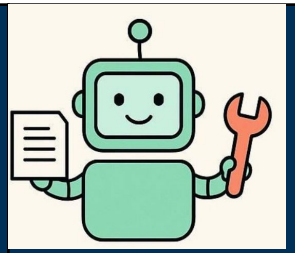
CS3300 Introduction to Software Engineering

# Lecture 12: Agentic Workflow and Project 2 Description

Dr. Nimisha Roy ▸ nroy9@gatech.edu

Slides and codebase adapted from Rola Dali

# From Generative to Agentic AI

|  Generative AI |  Agentic AI |
|---|---|
| Understands and **produces** content | Understands, produces **and acts on it** |
| Responds with text | Executes actions via tools or APIs |
| Focus = output generation | Focus = goal completion |
| Example: ChatGPT explains grading policy | Example: **GradeBuddy** computes averages and emails summaries |

# Gen AI, Agentic Systems and the Spectrum of Autonomy

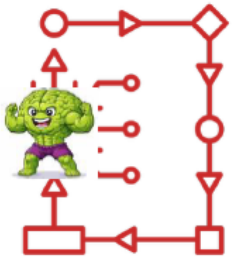
LLM
single call



LLM
in a loop
Chatbot



Workflow



Agent



Deep Agent



Autonomous
Agentic
Systems



AGI

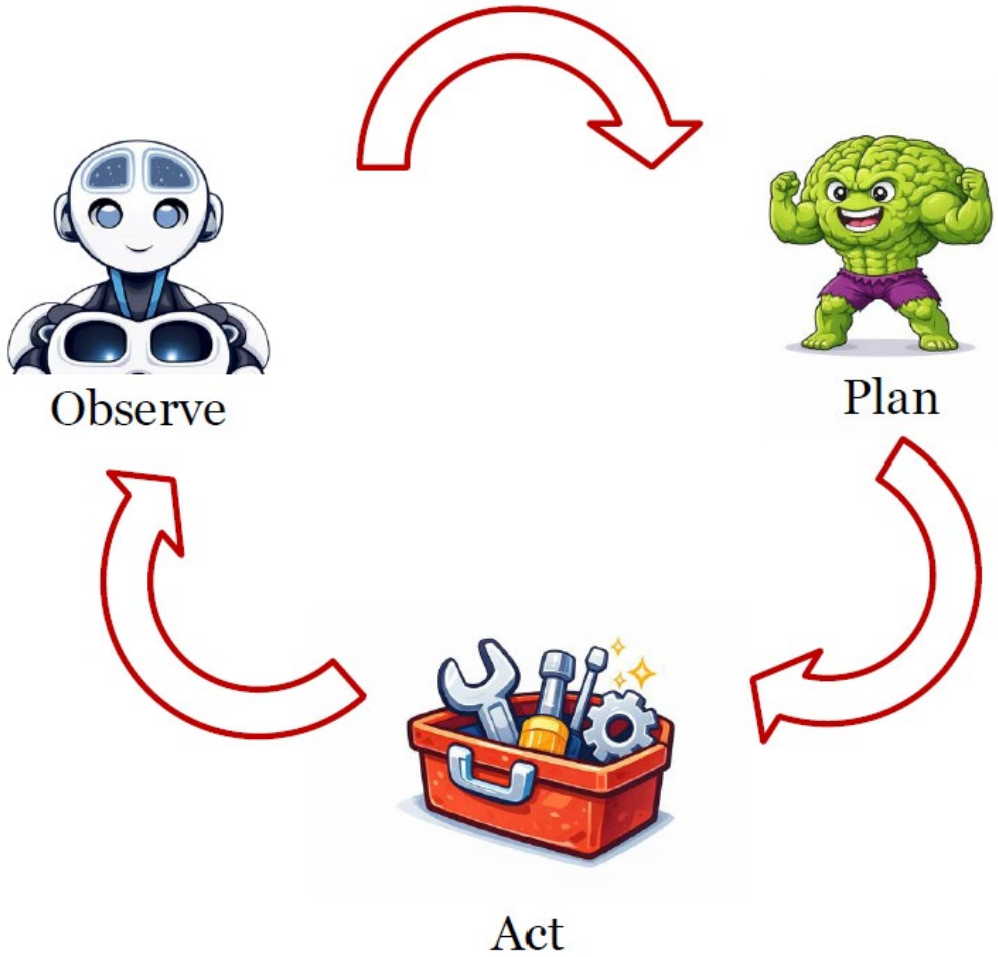


Agent Autonomy

Human Control

# What is an Agent?

An Gen AI agent is a software entity designed to perceive its environment, make decisions and take actions to achieve specific goals.




**Agent pseudocode**

```
while True:
    user query = get_user_input()
    response = invoke_llm(user_query)

    while response.has_tool_call():
        tool_result = invoke_tool(response.tool_spec)
        memory = append_context()
        response = invoke_llm(tool_output, memory)

    final_answer = response
    return final_answer
```

# Agent vs Workflow

## Task

- Book a travel activity
  - Check available activities
  - Check activity availability
  - Check calendar availability
  - Book/Pay for activity
  - Add activity to Calendar

## Workflow



Activities popular in Montreal?

activity_availability() | calendar_availability()



Loop till a desired activity fits availability

book_activity()

update_calendar()

## Agent



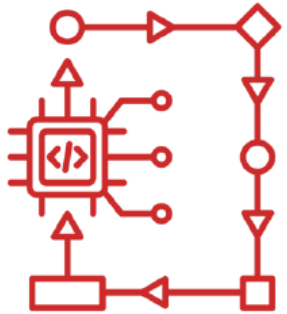
You are an activity booking agent.
Choose activities based on customer preferences
Book an available activity & update calendar



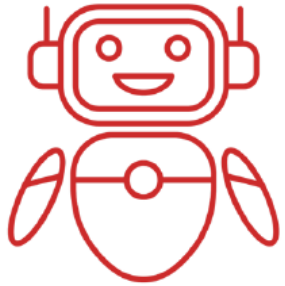
activity_availability()
calendar_availability()
book_activity()
update_calendar()

Agent ~ dynamic control flow devised by an LLM at Runtime
Workflows: static, pre-defined coded graphs

# Agent vs Workflow



- Mission Critical/Error Sensitive
- Regulated Industries/Deterministic outcomes
- Latency Sensitive
- Cost Sensitive
- Performance
- Flexibility
- Model driven Decision Making



# When to use, or not to use an agent

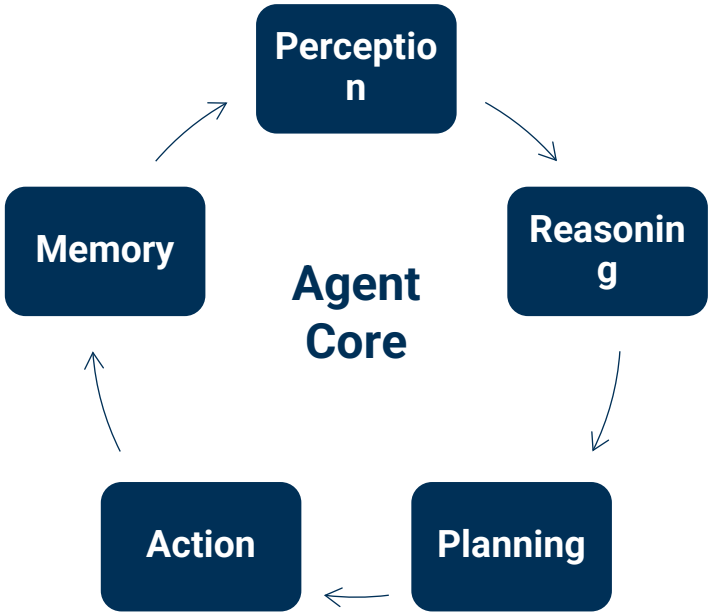
- Is the application: mission critical/error sensitive or in a highly regulated industry?
- Is the task path predictable or can be predefined?
- Is the value of the task worth the cost?
- Is the latency critical?

Use an agent in cases where error is tolerable, open-endedness is appreciated, the execution path is harder to code, cost is not an issue and latency can be tolerated

# Anatomy of an Agent

***GradeBuddy:*** *an AI agent that helps instructors summarize grades and send feedback automatically.*

| Component | Purpose | *GradeBuddy* Example |
|---|---|---|
| **Perception/ Goal** | Read and interpret inputs | Parse CSV grade files |
| **Reasoning/ Observe** | Decide what needs to be done | "Compute average before sending summary" |
| **Planning** | Sequence the steps logically | Retrieve → Analyze → Summarize → Email |
| **Action** | Execute tools and commands | Run calc_stats() and send_email() |
| **Memory** | Retain useful context over time | Recall previous semester's summaries |




Tier 2: Guardrails, Communication, Learning

# Types of Agentic Systems

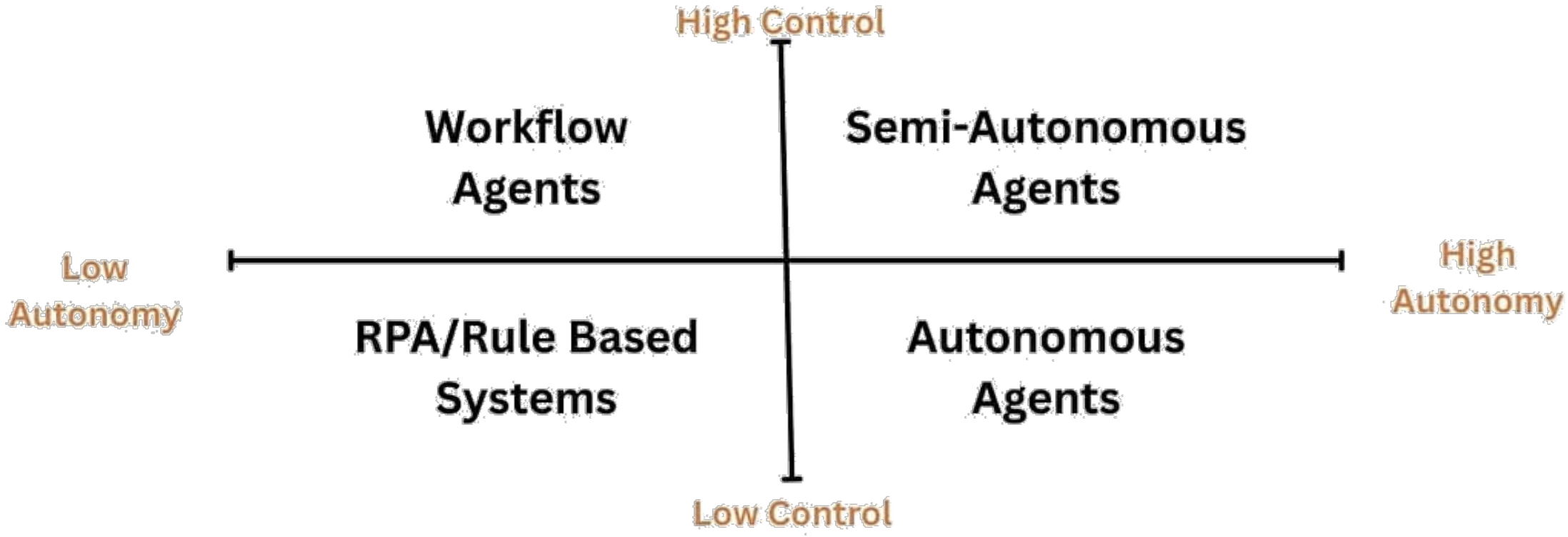


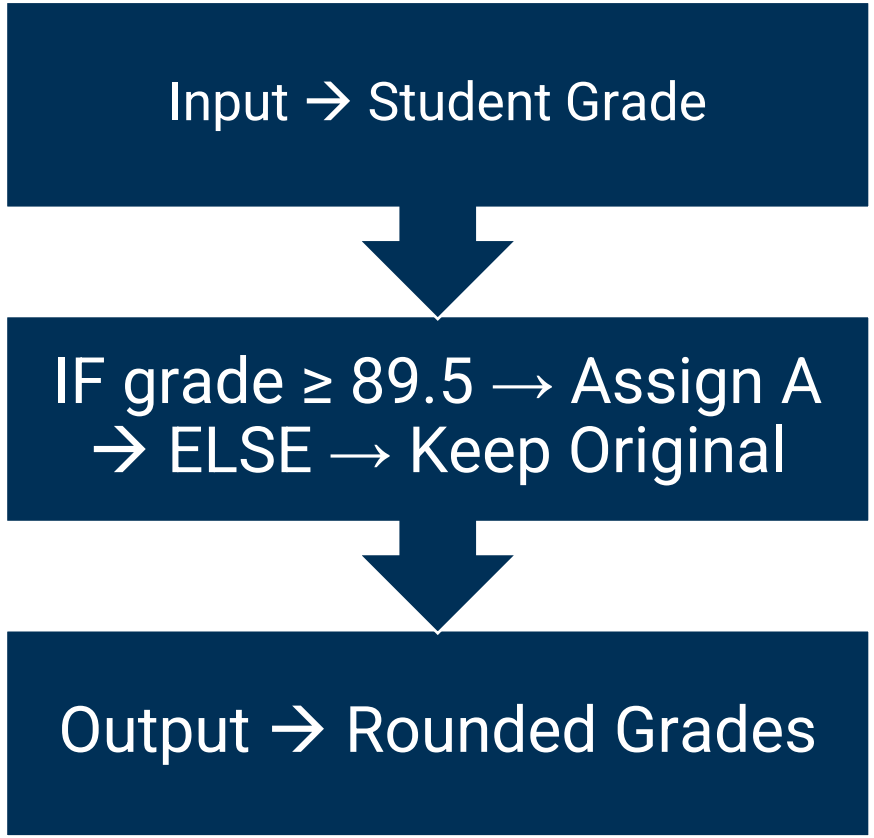

*Image credit: https://ai.plainenglish.io/types-of-agentic-systems-and-when-to-use-what-c49a1d601ccc*

# Rule-Based Agents

- Operate purely on if–then rules; no reasoning or learning.
- Best for structured, repetitive tasks with predictable outcomes.
- Fast, reliable, auditable, but brittle to change.

Example:

- *GradeBuddy* auto-rounds all grades ≥ 89.5 → 90.
- No adaptation — just executes the rule every time.




***GradeBuddy:*** *an AI agent that helps instructors summarize grades and send feedback automatically.*

# Workflow Agents

- Combine human judgment with AI assistance.
- AI performs a subtask — the human reviews and approves.
- Common in enterprise and applied NLP workflows.
- Reliable and low-risk, but requires supervision.

Example: *GradeBuddy* drafts feedback emails for students, but the instructor reviews and sends them.

| Lane | Steps |
| --- | --- |
| **AI Agent** | *Perception* → reads grade CSV → *Reasoning* → identifies trends → *Planning* → drafts messages → *Action* → outputs draft |
| **Human Instructor** | *Receives draft* → *Reviews content* → *Approves or edits* → *Sends* |

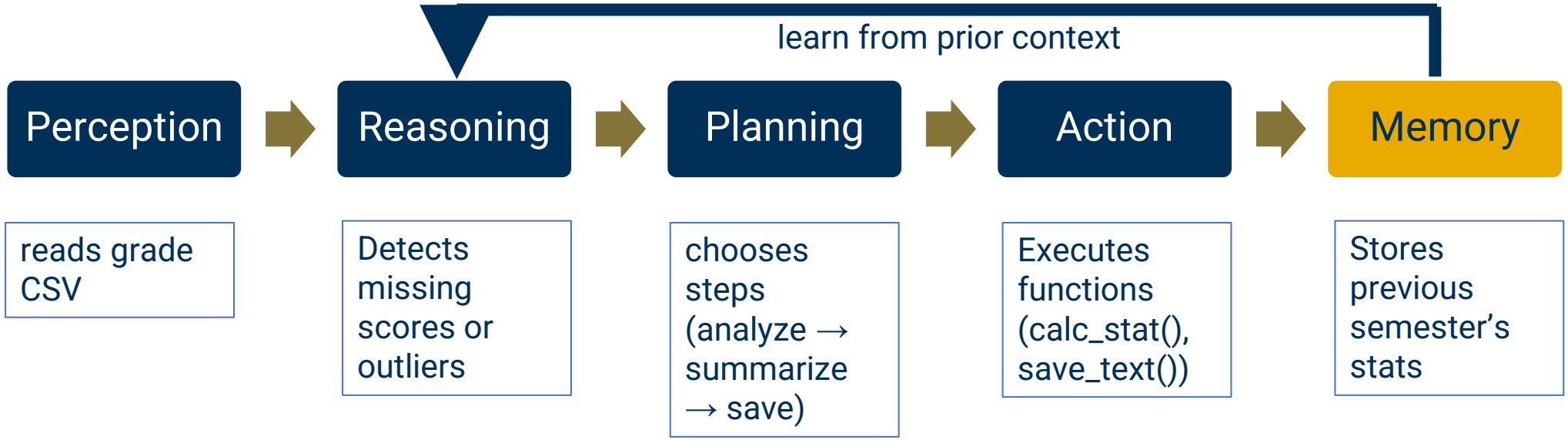


# Semi-autonomous Agents

- Can plan and execute multi-step workflows with minimal supervision.
- Use tools, reasoning, and limited memory to complete goals.
- Still provide checkpoints or human review triggers for safety.

Example: *GradeBuddy* retrieves grade data → computes averages → generates summaries → saves drafts automatically.

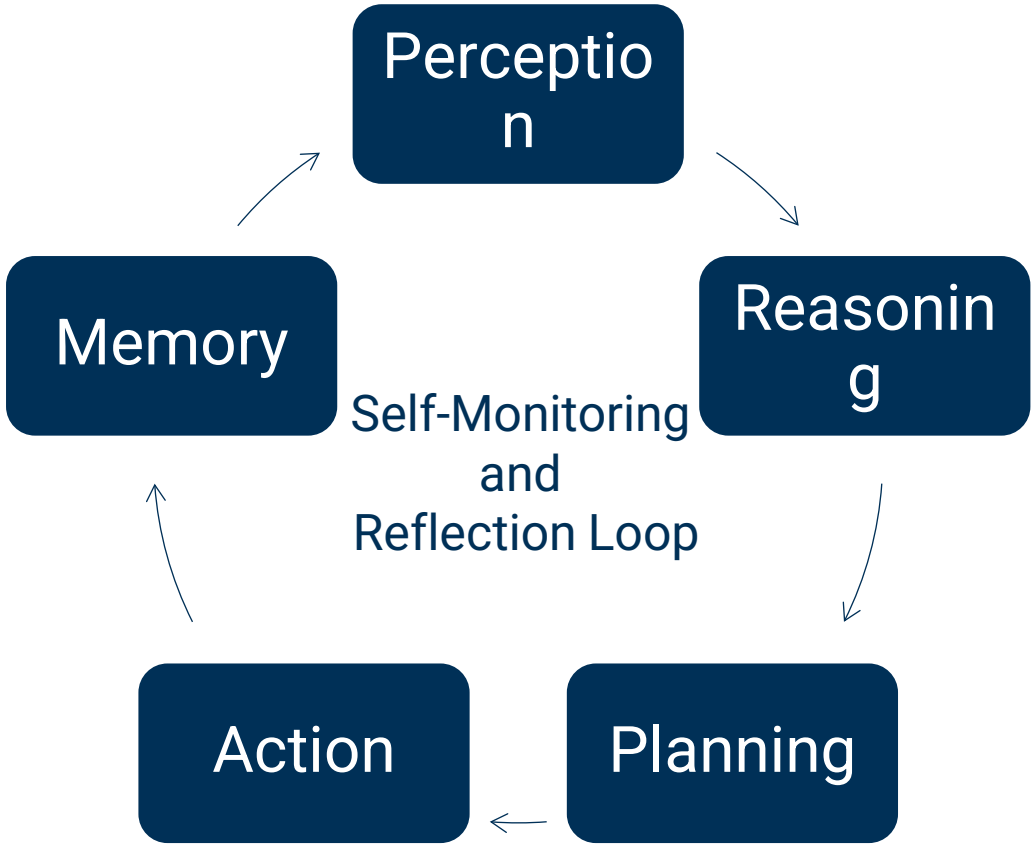
- Instructor only reviews final output before publishing.



# Autonomous Agents

- Can **plan, act, and adapt** without human supervision.
- Use **long-term memory, reasoning, and planning loops** to achieve goals.
- **Self-evaluate** and retry if actions fail.
- Require **guardrails** to prevent undesired or unsafe behavior.

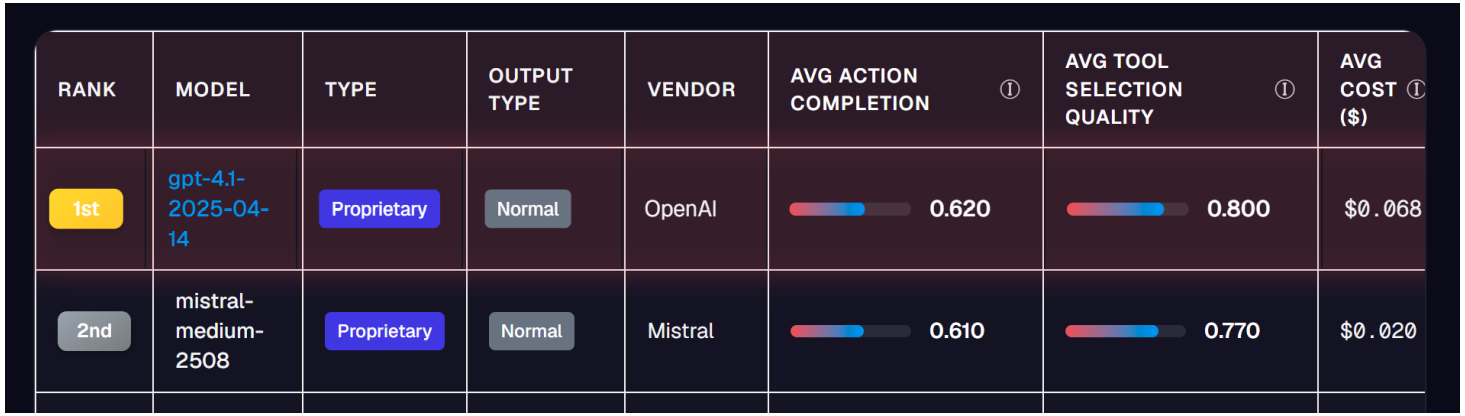
Example: *GradeBuddy* runs nightly to generate, verify, and email reports — it checks for data changes and sends new summaries automatically.



# Designing Agents - LLM

Criteria to consider when choosing a model:

- Task complexity
- Reasoning capabilities
- Context window size
- Tool Calling
- Latency
- Cost
- Compliance & data privacy



| RANK | MODEL | TYPE | OUTPUT TYPE | VENDOR | AVG ACTION COMPLETION ⓘ | AVG TOOL SELECTION QUALITY ⓘ | AVG COST ($) ⓘ |
|---|---|---|---|---|---|---|---|
| 1st | gpt-4.1-2025-04-14 | Proprietary | Normal | OpenAI | 0.620 | 0.800 | $0.068 |
| 2nd | mistral-medium-2508 | Proprietary | Normal | Mistral | 0.610 | 0.770 | $0.020 |

Gradebuddy – reliable tool calling, instruction following

# Designing Agents - Prompt

| Prompting Checklist | What to Include | Example |
| --- | --- | --- |
|  **1. Define the Role & Objective** | • Specify who the agent is.<br>• Clarify the goal, purpose, and success criteria. | **Example:** You are a helpful financial advisor. Your goal is to provide clear and actionable financial guidance. |
|  **2. Provide Necessary Context** | • Give relevant background information.<br>• Share any user context, constraints, or past interactions. | **Example:** The user is a college student with a part-time job and wants to start saving for the future. |
|  **3. Set Boundaries & Constraints** | • Define rules, limitations, and guardrails.<br>• Specify what the agent should avoid or not do. | **Example:** Do not provide investment advice. Avoid making assumptions about the user's income. |
|  **4. State the Desired Output Format** | • Define how the agent should respond.<br>• Specify structure, tone, length, or formatting. | **Example:** Respond in a friendly tone with bullet points and a short summary at the end. |
|  **5. Iterate and Refine** | • Test and observe agent behavior.<br>• Refine prompts based on results and feedback. | **Example:** If responses are too long, add: "Keep answers concise and under 150 words." |

# Designing Agents - Memory

- Memory allows agents to **retain and reuse context** across steps or tasks.
- Types of memory in agentic systems:
  - **Short-Term Memory:** Holds current conversation or reasoning context.
  - **Long-Term Memory:** Stores facts, retrieved documents, or historical insights.
  - **Episodic Memory:** Records past runs, decisions, and reflections for learning.
- Enables **multi-step reasoning** by connecting past actions with new goals.
- Implemented via **vector databases, knowledge graphs, or custom logs**.

# *GradeBuddy:* Memory in Action

*GradeBuddy* retains prior grading and feedback data to make reasoning iterative.

- **Short-Term Memory:**
  Stores the current course's grade distribution and student feedback context.
- **Long-Term Memory:**
  Keeps summaries from previous semesters and feedback templates that worked well.
- **Episodic Memory:**
  Logs each grading cycle ("what worked / what failed") to refine future actions.
- Enables **multi-step reasoning** across semesters — the agent gets "smarter" with experience.

# Designing Agents – Actions and Tools

- Tools give LLMs the ability to **interact with the world**. Enable agents to compute, retrieve, and store information.
- Agents can take several actions
  - Capability extension: calling functions, APIs, …
  - Knowledge Augmentation: retrieving data or context
  - Orchestration: calling other agents
- Tools can come in the form of



Function Call




API Call



Data retrieval



Browser Action

Example: *GradeBuddy* uses

- **Canvas API** → retrieve student submissions and grades
- **Python computation tools** → calculate averages and detect missing work
- **Outlook API** → email personalized feedback summaries
- **File storage (Dropbox / local logs)** → save reports for instructors



Code execution



Filesystem control

# Designing Agents - Guardrails and Safety Layers

- Guardrails define what an agent *can* and *cannot* do.
- Enforced across every stage: **Input** → **Plan** → **Tool** → **Output**.
- Common safeguards:
  - PII & content filters (to protect data and users)
  - Policy checks and rate limits (to control spend and access)
  - Human-in-the-Loop (HITL) for risky actions
  - Validation and schema checks before publishing
- Guardrails in *GradeBuddy* ensure feedback remains privacy-safe, neutral in tone, and policy-compliant before sending to students
- **Framework Examples**: NVIDIA NeMo Guardrails, Microsoft Presidio, Open Policy Agent (OPA), Guardrails AI, Azure AI Content Safety
- Builds trust, auditability, and compliance into AI workflows.

# The 5 Pillars of an Effective Agent

## 1 LLM (The Brain)

The language model that reasons, plans, and generates responses.

**Prompt Focus**
Define role, tone, style, and reasoning approach.

## 2 Memory (The Context)

Stores and retrieves past interactions, preferences, and important context.

**Prompt Focus**
Specify what the agent should remember or recall.

## Your Agent

Receives input → Thinks → Acts → Learns



## 3 Tools & Actions (The Hands)

Lets the agent interact with the world via APIs, search, code execution, etc.

**Prompt Focus**
Guide when and how the agent should use tools.

## 4 Guardrails (The Boundaries)

Ensures safe, ethical, and reliable behavior through rules, policies, and constraints.

**Prompt Focus**
Set limits, no-gos, and quality standards.

## Better Agent Outcomes

- Relevant
- Safe
- Reliable
- Context-aware
- Goal-oriented

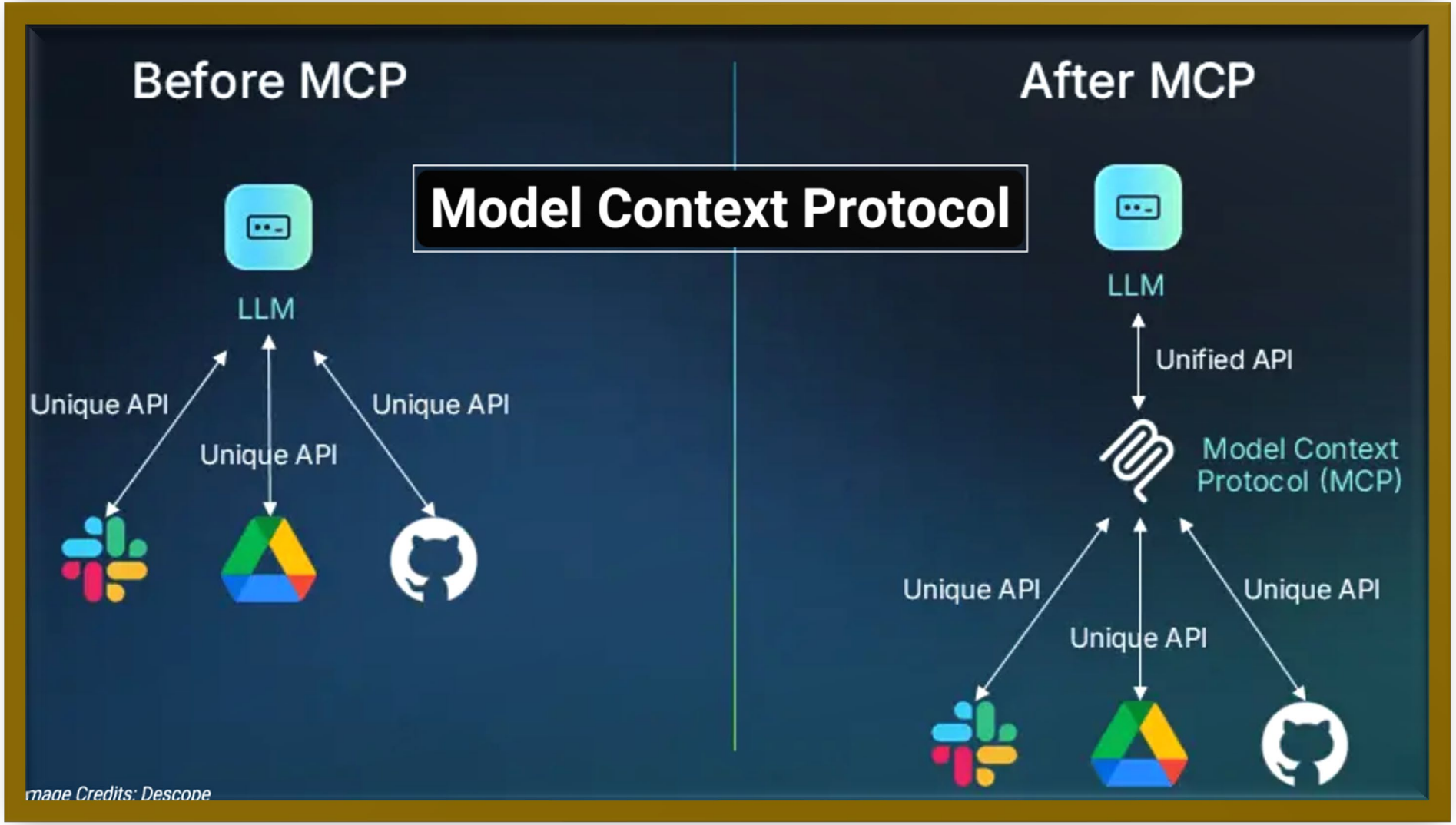
**Great prompting orchestrates all five.**
Clear instructions align the model, memory, tools, and guardrails to deliver accurate, helpful, and trustworthy outcomes.

### Prompting Checklist

- Define the role & objective
- Provide necessary context
- Specify tools & when to use them
- Set boundaries & constraints
- State the desired output format
- Iterate and refine

*AI-Generated Image*

# Model Context Protocol



# How MCP enables Agentic Architectures

**Model Context Protocol (MCP): A Standard Bridge Between LLMs and Tools**

- **Unified Interface:** Connects models to APIs, databases, and services through one protocol
- **Shared Context:** Passes memory, goals, and constraints across multiple calls or agents
- **Safety & Governance:** Applies authentication, rate-limits, and audit trails
- **Interoperability:** Works across frameworks like **CrewAI**, **LangGraph**, **AutoGen**, and **AWS AgentCore**
- **Impact:** Simplifies integration, reduces engineering overhead, and improves reliability for multi-agent systems

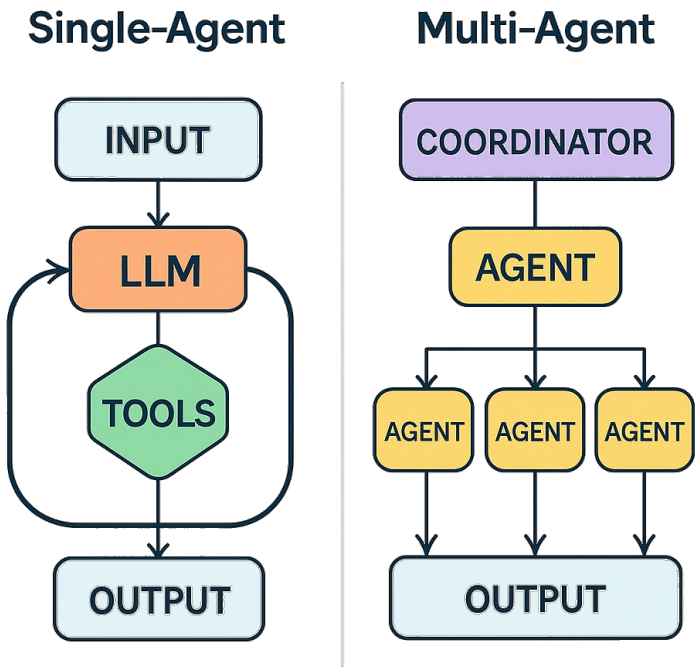
# From Single to Multi-Agent Systems

**Single-Agent Systems**

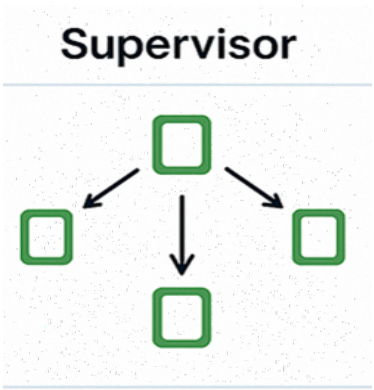
- One model handles perception → reasoning → action.
- Relies on **sequential processing** — one reasoning loop per query.

**Multi-Agent Systems**

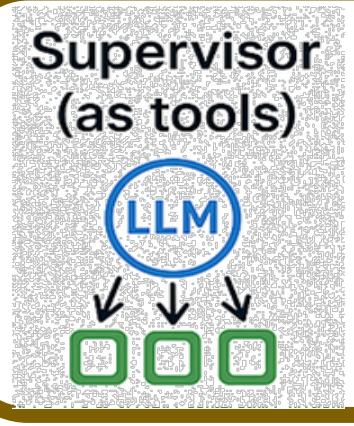
- Composed of **specialized agents** collaborating toward a shared goal.
- Each agent performs a role — Planner, Retriever, Executor, Evaluator.
- Coordination handled by a **central orchestrator** or peer-to-peer communication.
- Enables **parallelism**, scalability, and improved fault tolerance.



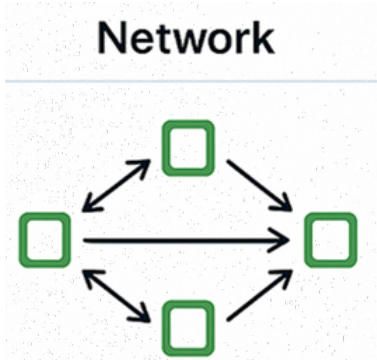
# Agentic Architecture Patterns



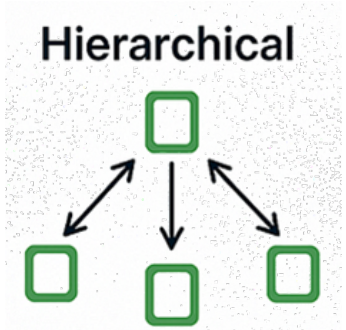

A central controller assigns and integrates outputs from sub-agents.



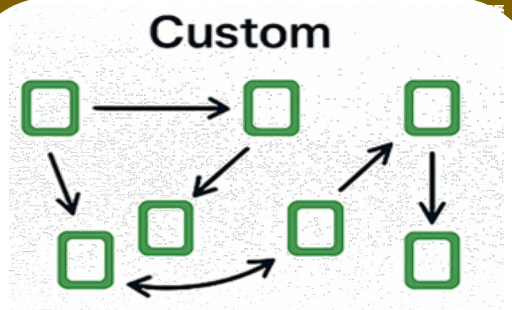

A planner invokes other agents as callable functions or APIs.




Agents communicate peer-to-peer to share results or validate outputs.




Multi-layer setup with sub-crews reporting to planners or leads

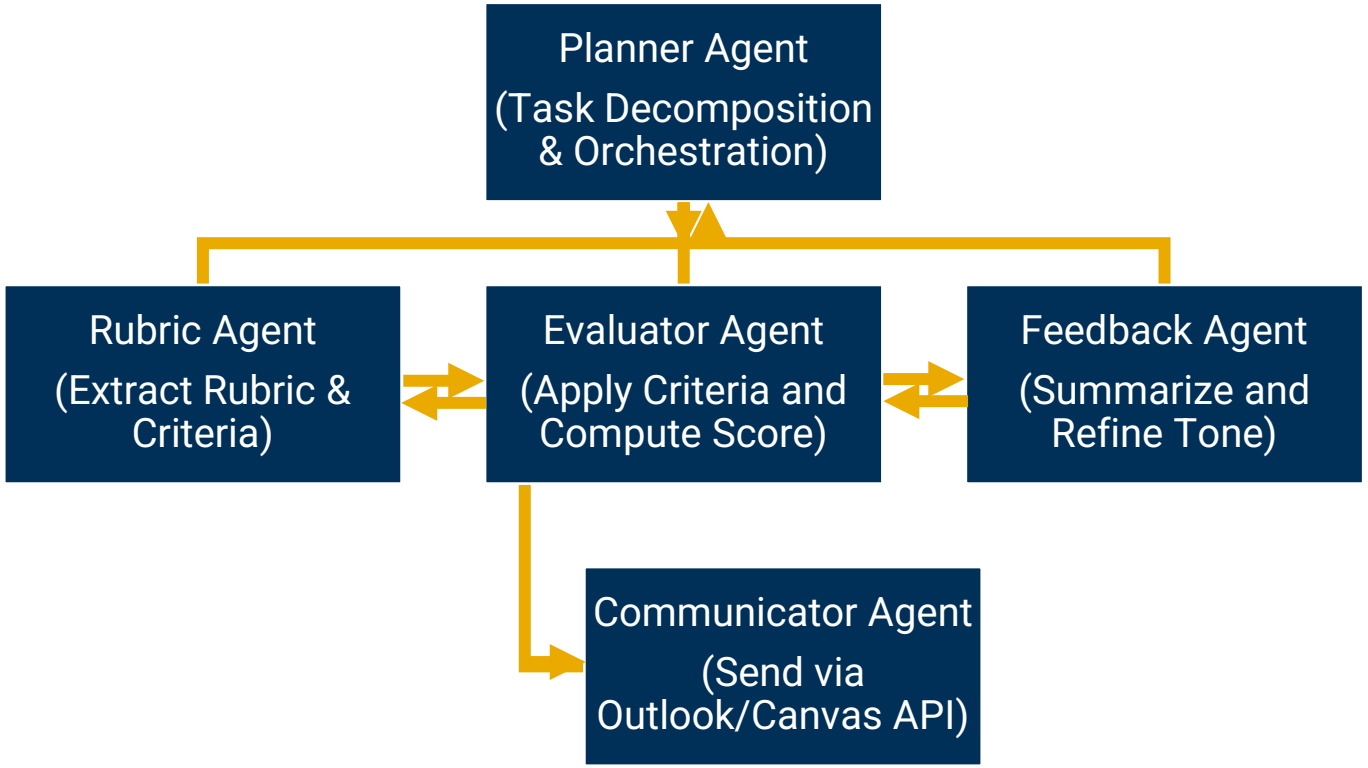



Adaptive and evolving links—agents self-organize based on task context.

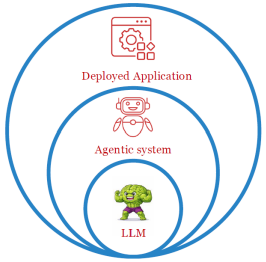
https://medium.com/@learning_37638/agentic-patterns-architectures-for-coordinated-ai-systems-34d9d8d8e1e2

# GradeBuddy Plus: Architecture Pattern & Guardrails

- **Coordination Pattern:**
  Hierarchical — Planner oversees all; agents report back structured results.
- **Guardrails:**
  NeMo Guardrails ensure that no sensitive data is exposed (compliance with FERPA) and that all responses follow instructional tone policies.
- **Continuous evaluation loops** verify that feedback is fair, consistent, and aligned with grading rubrics.



# Evaluating Agents




## LLM

- Following Instructions
- Task Completion
- Accuracy
- Hallucination
- Consistency
- Toxicity
- Guardrails

## Agentic System

- Did it achieve the goal?
- Did it choose an appropriate plan?
- Did it use the right tools and permissions?
- Were intermediate actions appropriate?
- Did it recognize failure and recover?
- Was human intervention needed at the right point?
- Was the result worth the cost/time/risk?

Two coding agents might both eventually fix the same bug. But one might inspect three files and run the appropriate tests, while another modifies twenty files and repeatedly retries failed actions. The final answer alone doesn't tell us enough.

# A Practical Agent Assessment Framework

| Dimension | Question to Ask | GradeBuddy Example |
|---|---|---|
| Outcome | Did it accomplish the goal correctly? | Correct grade summaries produced |
| Planning | Was its approach reasonable? | Calculated statistics before drafting feedback |
| Tool Use | Did it choose/use tools correctly? | Canvas read → calculation → email draft |
| Reliability | Does it behave consistently across runs/cases? | Handles missing grades and unusual cases |
| Safety & Security | Did it stay within permissions/constraints? | No unauthorized disclosure of student data |
| Efficiency | Was the trajectory unnecessarily expensive/long? | Avoids repeated API/tool calls |
| Human Oversight | Did it escalate when judgment was needed? | Instructor approves questionable feedback |

# Human in the Loop

AI is powerful. You bring the purpose, judgment, and responsibility.

### 1. THINK FIRST

Define the problem, goals, and what success looks like.



You set the direction. AI supports.

### 2. ASK & ITERATE

Prompt with context and intent. Refine based on results.



You guide the process. AI responds.

### 3. TAKE CHARGE

Decide what to use, adapt it, and take action.



You make decisions. AI provides options.

### 4. CHECK YOUR FACTS

Verify accuracy, interrogate assumptions, and check for bias.



You verify and validate. AI doesn't guarantee.

### 5. REFLECT & LEARN

Evaluate outcomes, learn what worked, and improve.



You learn and improve. AI gets better with you.

**Human-in-the-Loop Principles**

- You are accountable.
- You bring ethics and judgment.
- You ensure fairness and inclusion.
- You protect privacy and security.
- You keep the human impact front and center.

**AI can assist. You are in charge of your thinking, your judgment, and your impact.**

*AI-Generated Image*

# The Emerging Industry/Research Questions

**As agents become more autonomous:**

- **What should we assess?**
  Final outcome? Plan? Tool calls? Full trajectory? Cost? Security?
- **Who evaluates the evaluator?**
  Can another LLM reliably judge an agent, or do we still need deterministic tests and humans?
- **Where should the human be?**
  Every action? High-risk actions only? Exception handling? Final approval?
- **How do agents learn from feedback?**
  Feedback loops are not only about improving the agent—the human also learns how and when to delegate, intervene, and trust.
- **What changes with multiple agents?**
  Now failures can propagate between agents. Communication, orchestration, permissions, and trust boundaries become part of system correctness.
- **What happens when agents interact adversarially?**
  An agent can receive unsafe or malicious information through tools or other agents. Evaluation therefore has to include the **system around the model**, not just model capability.

# Implementing Agent - Demo

- Single LLM Call
- LLM Loop: ChatBot
- Simple Agent
- Agent with Memory
- Agent with LangChain
- LangChain Agent with Memory

# Project 2: Mandatory to Build AND Evaluate a Bounded Multi Agent

- **Goal** — What did you delegate and why?
- **Trajectory** — What plan/actions did the agents take?
- **Tool Use** — Which tools/APIs did they invoke? Were they appropriate?
- **Human Oversight** — Where did you intervene, approve, reject, or redirect?
- **Verification** — How did you establish that the result **and important intermediate actions** were correct?
- **Reflection** — Was agentic implementation actually better than a deterministic workflow?

# Project 2 Topic

Open Ended

Pick your own topic:

You need to justify that the topic is interesting, relevant to the course, and is of suitable difficulty

- Don’t have a project topic similar to project 1

Should have atleast 4 Minimum Marketable Features

# Technologies you are now familiar with

- Google Cloud
- Java Servlet
- REST Platform like Spring Boot
- Frontend development (js,html, css, possibly React…)
- Backend testing
- Debugging
- Working efficiently with IDEs, VCs
- AI tool incorporation in code generation, completion, and UML design

# Technologies for Project 2

Anything you want. Can be web-based or android application.

**Backend**: Build up your expertise in Java/SpringBoot or go for Node.js/others
**Frontend**: Keep it simple or try something new. React/Angular
**Database**: Datastore/FireBase/MySQL/mongo
**Testing**: More points allotted to testing (blackbox/whitebox) in Project 2

**Mandatory**: GitHub (PR)
**Deployment**: Google Cloud services to deploy. Build on your cloud expertise.

Bonus Points will be awarded to teams adopting interesting/difficult topics/technologies

# Project 2: Requirements

- Should have atleast 4 Minimum Marketable Features

- We will announce bonus points criteria soon
  - Completing AI survey at end of semester or extensive blackbox testing
  - Containerization/docker
  - More than 4 MMFs/Difficult/Innovative topic
  - Extensive tools or AI integration accomplished in whitebox and blackbox testing along with unit, integration and system testing.

- Have to use GitHub
  - PR and readme requirements same as project 1
- Last Assignment  - Test is based on Project 2. So focus on thorough testing

# Mandatory for Project 2

- 4 MMFs – get approval from mentors in project 2 touchpoint
- 1 Multi-agent workflow for a bounded task – discuss with mentor during next Friday's check-in
- Code Review Requirements
- Compliance with 3 design patterns (Which pattern and why applicable?)
  - Applicable to Python, Java, Kotlin, Scala, C#, Ruby, PHP
  - Inform the Instruction team on Ed if your team is attempting functional programming (Haskell, Erland, F#) or procedural programming (C, Pascal, in which case this requirement for your team will be revised accordingly
  - Even if you are only using JS, TypeScript and ES6 classes support classic OOP patterns
- Good testing done – blackbox and whitebox.
  - Relevant to testing assignment and project 2 ppt
- GitHub
- GCP
- AI integration – implementation (atleast 1 tool based on Project 1 experience) and testing (research with different tools)
  - Discuss plan with mentors next Friday in class

# GitHub

- Make sure to have your GitHub repository set for this project
- Ensure it is private
- It is important for your future reference
- Add it to your resume
- GitHub pages can be very compelling for employers
  - You will use GitHub pages to create your Project Report

# Project 2 Topic and MMF assignment

# MMF and MVP

Substantial piece of functionality that delivers business value to customers

- Should comprise multiple small user stories

An MVP comprises several MMFs

For LocalLens, some MMFs were:

- Basic Display – Input and Output
- Interactive Map Display – hover functionality
- Advanced Sorting, Searching and Filtering functionalities
- Change Map pin color for example is not a MMF